



What Are Sets, and How Do They Work?

Sets are one of Python's built-in data structures. One of the core characteristics of sets is that they don't store duplicate values. If you try to add a duplicate value to a set, only one of them will be stored.

Sets are mutable and unordered, which means that their elements are not stored in any specific order, so you cannot use indices or keys to access them. They can only contain values of immutable data types like numbers, strings, and tuples. And they support mathematical set operations, including union, intersection, difference, and symmetric difference.

To define a set, you just need to write its elements within curly braces and separate them with commas. This is an example of a set of numbers:

```
my_set = {1, 2, 3, 4, 5}
```

One quirk of working with sets is that, if you ever need to define an empty set, you must use the `set()` function. If you just write empty curly braces, like `{}`, Python will automatically create a dictionary.

```
set() # Set
{}    # Dictionary
```

You can add an element to a set with the `.add()` method, and pass in the new element as argument:

```
my_set.add(6)
```

In our example, the new set would be:

```
{1, 2, 3, 4, 5, 6}
```

If you try to add an element that is already in the set, only one will be kept. In this case, we already have the number 5 in the set:

```
my_set.add(5)
```

So the set will not change:

```
{1, 2, 3, 4, 5, 6}
```

To remove an element from the set, you have two options. You can either use the `.remove()` method or the `.discard()` method, and pass in the element that you want to remove as argument.

The `.remove()` method will raise a `KeyError` if the element is not found, while the `.discard()` method will not:

```
my_set.remove(4)
my_set.discard(4)
```

The `.clear()` method removes all the elements from the set:



Python sets also have powerful methods that perform common mathematical set operations.

The `.issubset()` and the `.issuperset()` methods check if a set is a subset or superset of another set, respectively.

Here, we are checking if `your_set` is a subset of `my_set`, which is `False` because not all the elements of `your_set` are in `my_set`.

We are also checking if `my_set` is a superset of `your_set`. This is also `False` because `my_set` does not have all the elements of `your_set`:

```
my_set = {1, 2, 3, 4, 5}
your_set = {2, 3, 4, 6}

print(your_set.issubset(my_set)) # False
print(my_set.issuperset(your_set)) # False
```

The `.isdisjoint()` method checks if two sets are disjoint, which means they don't have any elements in common. In this case, that's `False` because `my_set` and `your_set` do have common elements – 2, 3, and 4:

```
print(my_set.isdisjoint(your_set)) # False
```

The union operator `|` returns a new set with all the elements from both sets:

```
my_set | your_set # {1, 2, 3, 4, 5, 6}
```

The intersection operator `&` returns a new set with only the elements that the sets have in common:

```
my_set & your_set # {2, 3, 4}
```

The difference operator `-` returns a new set with the elements of the first set that are not in the other sets. In this example, the numbers 1 and 5 are in `my_set` but NOT in `your_set`:

```
my_set - your_set # {1, 5}
```

The symmetric difference operator `^` returns a new set with the elements that are either in the first or the second set, but not both. In this case, 1 and 5 are in `my_set` but not in `your_set`, so they are included. And the number 6 is in `your_set` but not in `my_set`, so it's included as well:

```
my_set ^ your_set # {1, 5, 6}
```

Each one of these operators also has its corresponding compound assignment operator if you add the equal sign next to it. These operators automatically assign the resulting set to the first set in the expression:

```
|=  &=  -=  ^=
```

For example, the `-=` operator finds the difference between the sets and updates the first set with that result:

```
my_set -= your_set
```



```
print(my_set) # {1, 2, 3}
```

You can check if an element is in a set or not with the `in` operator. Here, we are checking if 5 is in `my_set`. The result will be a boolean value `True` or `False`:

```
print(5 in my_set)
```

And those are the fundamentals of sets. They are very helpful when you don't need to store the values in any specific order, and when you only need to store unique values.

Questions

Which of the following is a core characteristic of Python sets?

Elements are ordered and accessed by index.

Elements are stored as key-value pairs.

Elements are unique and unordered.

Correct!

Elements can be of any data type, including lists and dictionaries.

What operator is used to check if an element is present in a set?

`==`

`in`

Correct!

`get()`

`find()`

Which set operation returns a new set with the elements that are present in either one of the two sets, but not in both of them?

Union



Python Certification

Working with Dictionaries and Sets

Difference

Symmetric Difference

Correct!

Check your answer

Ask for Help